

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет ИСТ
Кафедра Информационные системы
Дисциплина Программирование мобильных устройств

КУРСОВОЙ ПРОЕКТ (РАБОТА)

Тема Разработка мобильного приложения «Каталог манги» на Flutter для Android

Выполнил студент _____ / Разубаев С.М. /
подпись инициалы, фамилия

Курс 3 Группа Пибд-31

Направление/ специальность Програмная инженерия

Руководитель доцент, к.т.н, доцент
должность, ученая степень, ученое звание
Филиппов Алексей Александрович
фамилия, имя, отчество

Дата сдачи:
« » 20 г.

Дата защиты:
« » 20 г.

Оценка:

« » 20 Г

ОТЗЫВ
руководителя на курсовой проект (работу)

фамилия, имя и отчество

Тема проекта (работы) Разработка мобильного приложения «Каталог манги» на Flutter для Android

[illegible]

« » 2024 г.

Введение

В современном мире цифровых технологий манга становится все более популярной среди различных возрастных групп и культур. Множество людей по всему миру привлечены разнообразными историями и уникальными персонажами. С ростом интереса к этому искусству, появляется необходимость в удобных инструментах для доступа и организации коллекции манги.

Существует множество онлайн-ресурсов и баз данных, которые содержат информацию о манге. Но многие из них не удобно использовать на мобильном приложении.

В рамках данной работы разрабатывается мобильное приложение "Каталог манги", которое предоставляет пользователю возможность легко находить интересующие его произведения, изучать их жанры, авторов и сохранять понравившиеся для дальнейшего просмотра.

В данной работе представлено краткое описание предметной области, выделены основные сущности и бизнес-процессы, подлежащие автоматизации.

Были выделены следующие сущности:

- Манга
 - Японская форма комиксов и графических новелл, каждая единица которого имеет название, изображение, описание и рейтинг.
 - Сущность манга формирует основной набор данных, который отображается в приложении.
- Карточка манги

- Графический элемент в интерфейсе приложения, который представляет собой сокращенную информацию о манге (изображение, название, статус и рейтинг).
 - Позволяет пользователю быстро ознакомиться с содержанием и выбрать интересующий объект для более детального изучения.
- Лайк
 - Механизм, который позволяет пользователю отмечать произведения, которые ему понравились
 - Информация о лайках сохраняется локально для дальнейшего доступа.

Используются следующие бизнес-процессы:

- Отображение каталога манги

Приложение должно автоматически запрашивать данные из API и выводить их в виде карточек. Каждая карточка содержит информацию о произведении, которая состоит из обложки, названия, статуса и рейтинга. Это дает возможность пользователю быстро просматривать доступные варианты.

- Поиск и фильтрация данных

Пользователь может использовать поиск манги по названию для нахождения конкретных произведений. Поисковые запросы отправляются через API, после чего приложение отображает результаты.

- Просмотр произведения

При выборе карточки манги пользователь переходит на страницу с информацией о выбранном произведении

- Добавление в понравившееся

Для персонализации опыта пользователям предоставляется возможность ставить отметку "мне нравится". После установки лайка данные сохраняются в локальном хранилище, чтобы пользователь мог позже вернуться к своим предпочтениям.

- Локализация и настройка приложения:

Приложение поддерживает несколько языков, что делает его доступным для широкой аудитории. Также изменена стандартная иконка приложения для соответствия тематике.

- Обновление данных:

Пользователь может вручную обновлять список данных для получения актуальной информации.

Глава 1. Проектирование программной системы

1.1. Информационное обеспечение

В разрабатываемом приложении используются следующие сущности:

- HomeData
- CardData

ER-диаграмма сущностей:

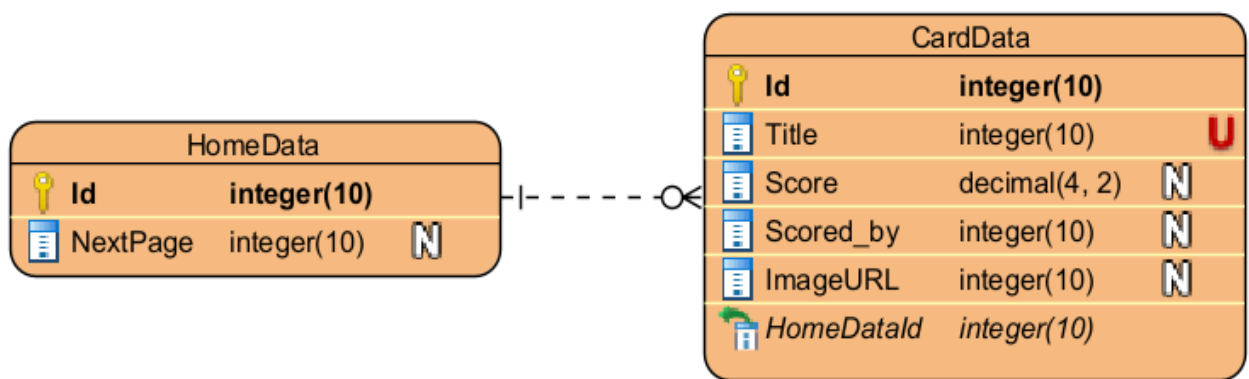


Рис. 1 ERD-диаграмма

Подробное описание каждой сущности:

1. CardData

1.1.Id

1.1.1. Уникальный идентификатор

1.1.2. Число

1.2.Title

1.2.1. Название манги

1.2.2. Строка

1.3. Score

1.3.1. Рейтинг манги

1.3.2. Дробное число с 2 знаками после запятой

1.4.Scored_by

1.4.1. Количество людей, оценивших произведение

1.4.2. Число

1.5.ImageURL

1.5.1. Ссылка на обложку манги

1.5.2. строка

1.6.HomeDataId

1.6.1. Ссылка на HomeData

1.6.2. Число

2. HomeData

2.1. Id

2.1.1. Уникальный идентификатор отображаемой страницы

2.1.2. Строка

2.2. NextPage

2.2.1. Номер следующей страницы

2.2.2. Число

Связь между сущностями:

HomeData – CardDate – Связь один ко многим. В одном HomeData может храниться множество элементов сущности CardDate.

1.2.Интерфейс пользователя

Интерфейс содержит в себе 2 различных экрана, содержащих различную информацию:

1. Главный экран

1.1. Экран, состоящий из следующих элементов:

1.1.1. Строка поиска

1.1.2. Кнопка, предназначенная для смены локализации

1.1.2.1. Выглядит как иконка флага, которая отображает выбранный язык

1.1.3. Список карточек произведений. Каждая карточка состоит из следующих элементов:

1.1.3.1. Обложка

1.1.3.2. Название произведения

1.1.3.3. Статуса

1.1.3.4. Рейтинга

1.1.3.5. Количества людей, поставивших оценку

1.1.3.6. Кнопка лайка

1.1.3.6.1. Первоначально выглядит как сердечко в виде с черной окантовкой и белой заливкой. При нажатии на лайк заливка и окантовка иконки становятся красными

2. Экран с подробной информацией о произведении

2.1. Кнопка перехода на главный экран

2.2. Обложка

2.3. Название

2.4. Статус

2.5. Рейтинг

2.6. Количество людей, оценивших произведение

Изображения экранов:

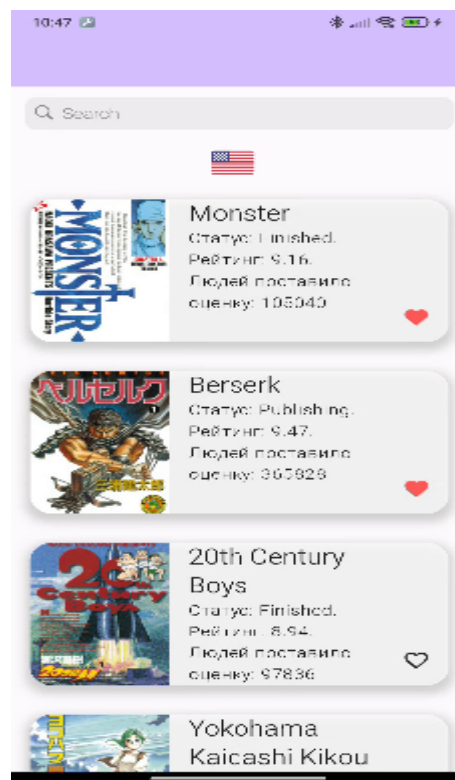


Рис. 2 Главный экран с локализацией на английском языке

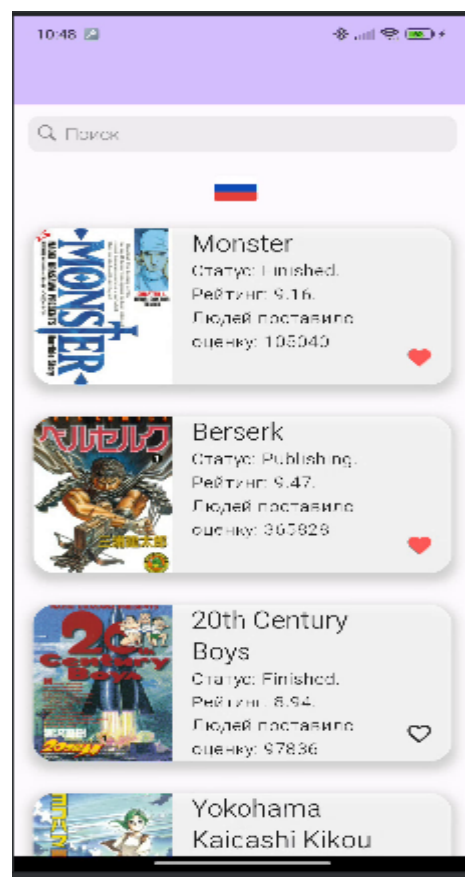


Рис. 3 Главный экран с локализацией на русском языке

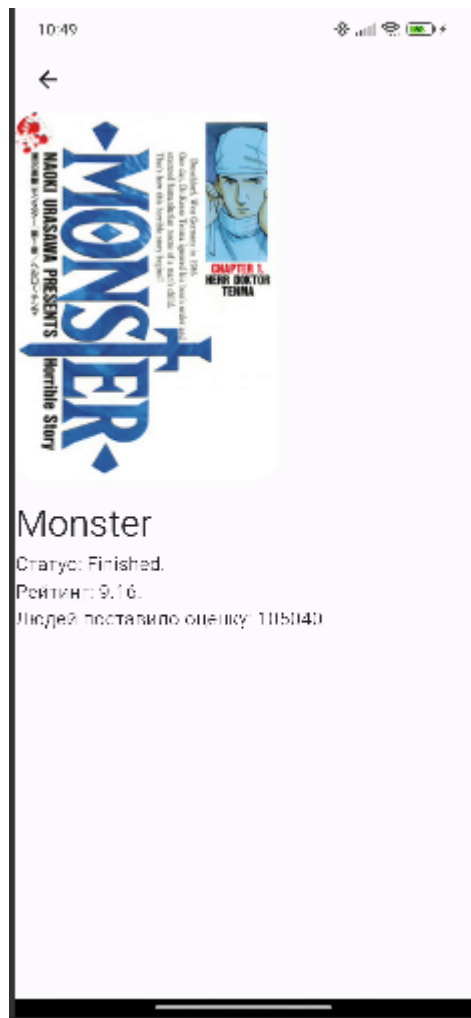


Рис. 4.Экран с подробной информацией о произведении



Рис 5. Карта экранов приложения

Пользователь может переходить с главного экрана на экран с подробной информацией. И наоборот, с экрана с подробной информацией переходить на главный экран.

1.3. Архитектура приложения

Приложение разработано с использованием многослойной архитектуры и паттерна BLoC (Business Logic Component), который отделяет бизнес-логику от

пользовательского интерфейса. Это способствует упрощению управления состоянием, тестирования и дальнейшему развитию приложения.

Особенности архитектуры приложения

1. Модульность:

Архитектура BLoC способствует созданию модульных компонентов, что позволяет легко переиспользовать логику в разных частях приложения или даже в разных проектах.

2. Многослойность

Каждый слой имеет свою определенную ответственность, что упрощает разработку и поддержку приложения. Например, слой представления отвечает только за отображение данных, а бизнес-логика обрабатывает все правила и операции.

3. Четкая организованность

BLoC обеспечивает ясную структуру кода, деля его на компоненты, такие как события (Events), состояния (States) и сам BLoC. Это улучшает читаемость и поддерживаемость кода.

4. Поддержка многопоточности

BLoC позволяет обрабатывать события и состояния в отдельных потоках, что может повысить производительность приложения

Слои архитектуры и их функциональное назначение:

1. Слой представления (UI Layer):

- Отвечает за отображение данных и взаимодействие с пользователем
- Отображает интерфейс(экраны, карточки, кнопки и т.д)
- Основные функции:
 1. Отображение списка карточек.
 2. Реакция на действия пользователя

3. Навигация по карте экранов

- Взаимосвязь с другими слоями:

1. Отправляет события в слой управления состоянием (BLoC).
2. Получает состояния и данные от слоя управления состоянием для обновления интерфейса.

2. Слой управления состоянием (BLoC Layer):

- Содержит логику приложения: обработку пользовательских событий, преобразование данных и управление состоянием.

- Основной функционал:

1. Запрос данных из слоя данных через репозитории.
2. Фильтрация и преобразование данных.
3. Управление лайками и обновлением состояния UI.
4. Реализация Debounce для уменьшения количества запросов при поиске.

3. Слой доступа к данным (Data Layer):

- Отвечает за работу с внешними API и локальным хранилищем.

- Основной функционал:

1. Выполнение запросов к API и получение данных о карточках манги.
2. Сохранение локальных данных, таких как лайки и настройки, в SharedPreferences.
3. Преобразование данных в объекты (DTO) для передачи в слой управления состоянием.

- Компоненты слоя данных:

1. Data Sources:

- Remote Data Source — для взаимодействия с внешними API.

- Local Data Source — для работы с локальным хранилищем (SharedPreferences). Используется для сохранения понравившейся манги.

2. Repositories:

- Абстракция, объединяющая источники данных и предоставляющая интерфейсы для слоя бизнес-логики.

1.4. Программный интерфейс и функции серверной части

Приложение использует REST API для получения данных о списке манги. API предоставляет данные в формате JSON, которые затем преобразуются в объекты с помощью предоставленных DTO-классов.

Описание структуры и особенности REST API:

- Формат данных: JSON.
- Методы HTTP:
 - GET: используется для получения информации о списке манги с детальной информацией, а также о странице.
- Endpoints(пример если используется сторонний API):
 - /manga: возвращает список манги с возможностью фильтрации и пагинации

Функциональные возможности серверной части:

1. Получение списка манги

- Эндпоинт /manga.
- Поддерживаемые параметры запроса:
 - page: номер страницы для пагинации.
 - q: строка для поиска манги по названию.
 - limit: количество элементов на странице.

2. Пагинация и фильтрация

- API поддерживает пагинацию через параметры `page` и `limit`, а также поиск по названию через параметр `q`, что позволяет гибко управлять отображением данных в приложении.

Структура DTO:

DTO (Data Transfer Object) представляет собой объекты, используемые для получения и передачи данных между клиентом и сервером. В предоставленном коде определены следующие DTO:

1. MangasDto

- Описывает данные ответа, содержащего список манги.
- Поля:
 - `data`: список манги (`List<MangaDataDto>`).
 - `pagination`: данные о пагинации (`MangaPaginationDto`).

2. MangaDataDto

- Описывает данные о каждой манге.
- Поля:
 - `id`: идентификатор манги.
 - `title`: название манги.
 - `type`: тип произведения.
 - `status`: статус манги.
 - `scored_by`: количество людей, которые поставили оценку манге
 - `score`: рейтинг.
 - `images`: ссылки на изображения для обложек (`MangaDataImagesDto`).

3. MangaDataImagesDto

- Описывает набор изображений для манги.
- Поля:
 - `jpg`: изображения в формате JPG (`MangaDataImagesJPGDto`).

4. MangaDataImagesJPGDto

- Содержит ссылки на изображения в разных размерах.

- Поля:
 - `image_url`: ссылка на изображение.

5. MangaPaginationDto

- Описывает данные о пагинации.
- Поля:
 - `lastVisiblePage`: последняя доступная страница.
 - `currentPage`: текущая страница.
 - `hasNextPage`: есть ли следующая страница (true/false).

Глава 2. Разработка программной системы

2.1 Слои приложения

Описание основных пакетов:

1. Основные зависимости для реализации функционала:

- `shared_preferences`: используется для локального хранения данных, таких как лайки.
- `dio` и `pretty_dio_logger`: для работы с REST API и логирования запросов.
- `json_annotation`, `json_serializable`, `build_runner`: для автоматической сериализации и десериализации данных (например, DTO).
- `flutter_bloc`, `equatable`: для управления состояниями и событийной логики.
- `intl` и `flutter_localizations`: для реализации локализации.

2. Иконки и стилизация:

- `flutter_launcher_icons`: замена стандартной иконки приложения на кастомную (`assets/icon.png`).

3. UI и графические элементы:

- `flutter_svg`: для работы с SVG-графикой, поддерживающей отображение изображений в формате SVG.

4. Навигация между слоями:

- Используются методы разделения интерфейсов (DTO, репозитории) и структуры BLoC для инкапсуляции логики и связывания UI с данными.

Взаимодействие между слоями:

1. Пользователь вводит запрос в поисковую строку.
2. UI отправляет событие в BLoC, которое обрабатывается бизнес-логикой.

3. BLoC вызывает репозиторий, который отправляет запрос через dio к REST API.
4. API возвращает данные в формате JSON.
5. DTO преобразует JSON в объект, который репозиторий возвращает в BLoC.
6. BLoC обновляет состояние, и данные передаются обратно в UI для отображения.

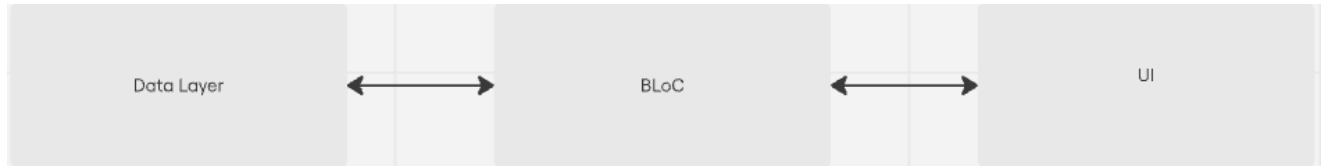


Рис 6. Схема взаимодействия между слоями

Основные классы и интерфейсы, работающие с хранилищем:

1. Интерфейс: ApiInterface

- определяет контракт для работы с API в приложении
- Методы:
 - `Future<HomeData?> loadData({OnErrorCallback? OnError});`
Определяет метод для загрузки данных. Реализация этого метода обязана возвращать доменную модель HomeData.

2. Основной репозиторий: MangaRepository

- Репозиторий реализует интерфейс ApiInterface и отвечает за получение данных из API. Основные элементы:
- Поля:
 - `_dio`: объект Dio для отправки HTTP-запросов.
 - `_baseUrl`: базовый URL API (<https://api.jikan.moe>).
- Методы:
 - `loadData`:
 1. Получает данные из API на основе переданных параметров и преобразует их в доменную модель HomeData.

- Параметры:
 - onError: коллбек для обработки ошибок.
 - q: строка поиска.
 - page: номер страницы.
 - pageSize: количество элементов на странице.
- Возвращаемое значение:
 - Экземпляр HomeData или null в случае ошибки.

3. Типы данных и доменные модели:

- CardData:
 - Модель данных для отображения карточек манги:
 1. text: название.
 2. descriptionText: описание.
 3. imageUrl: ссылка на изображение.
 4. id: уникальный идентификатор.
- HomeData:
 - Модель для главной страницы:
 1. data: список объектов CardData.
 2. nextPage: номер следующей страницы.

4. Мапперы (Extensions):

- Мапперы отвечают за преобразование DTO (Data Transfer Objects) в доменные модели (Domain Models). Они инкапсулируют логику преобразования и позволяют разделить данные, полученные из API, от внутренних структур приложения.
- Методы:
 - MangaDataDto.toDomain:
 1. Преобразует объект MangaDataDto в CardData:
 - MangasDto.toDomain:

1. Преобразует объект MangasDto в HomeData:

2.3 Слой для реализации бизнес-логики

Слой бизнес-логики (BLoC) отвечает за управление состоянием и реализацией бизнес-логики приложения. Он изолирует пользовательский интерфейс от взаимодействия с данными, что позволяет работать с репозиториями, обрабатывать информацию и передавать её в интерфейс.

Основные компоненты слоя бизнес-логики:

1. BLoC (Business Logic Component):

Этот компонент управляет событиями и состояниями приложения.

Наследуются от `Bloc<Event, State>`.

2. Events (События):

События представляют собой действия пользователя или изменения, которые необходимо обработать. .

3. States (Состояния):

Состояния определяют текущий статус приложения, который передается в пользовательский интерфейс. Наследуются от `Equatable`.

Содержат `get props`, необходимый для сравнения объектов.

Основные «блоки»:

1. HomeBloc:

Класс, который отвечает за отображение главной страницы приложения

1.1. HomeBloc:

1.1.1. Методы:

- `_onLoadData(HomeLoadDataEvent event, Emitter<HomeState> emit)`
 - Обработывает событие `HomeLoadDataEvent`, загружая данные из репозитория.

1.1.2. Поля:

- репо
 - Хранит MangaRepository

1.2.HomeEvent

1.2.1. HomeLoadDataEvent

- Поля:
 - Search
 - Строка запроса для поиска по названию
 - nextPage
 - Номер следующей страницы для пагинации

1.3. HomeState

1.3.1. Методы

- HomeState copyWith({HomeData? data, bool? isLoading, bool? isPaginationLoading, String? error})
 - Обновляет необходимые поля

1.3.2. Поля

- Data
 - Хранит в себе HomeData
- isLoading
 - Флаг, показывающий грузятся ли сейчас данные из API

2. LikeBloc:

2.1.LikeBloc:

2.1.1. Методы

- _onLoadLikes(LoadLikesEvent event, Emitter<LikeState> emit)
 - Обрабатывает событие LoadLikesEvent, подгружая из SharedPreferences информацию о том, какие произведения были отмечены как понравившиеся.
- _onChangeLike(ChangeLikeEvent event, Emitter<LikeState> emit)

- Обработывает событие `ChangeLikeEvent`, которое ставит/убирает лайк с манги.

2.2. LikeEvent

Все события наследованы от абстрактного класса `LikeEvent`

2.2.1. LoadLikesEvent

- Методы и поля отсутствуют

2.2.2. ChangeLikeEvent

- Поля
 - `Id`
 - Идентификатор манги, которому нужно поставить/убрать лайк.

2.3. LikeState

2.3.1. Поля:

- `List<String>? likedIds`
 - Список идентификаторов манги, которая понравилась

3. LocaleBloc

3.1. LocaleBloc

3.1.1. Метод:

- `_onChangeLocale(ChangeLocaleEvent event, Emitter<LocaleState> emit)`
 - Обработывает событие `ChangeLocaleEvent`, заменяющий текущую локализацию на следующую из существующих. (В приложении присутствует английская и русская)

3.2. LocaleEvent

3.2.1. ChangeLocaleEvent

- Не содержит методов и полей

3.3. LocaleState

3.3.1. Поля

- Locale currentLocale
 - Текущая локализация

2.4 Слой представления

1. Основные классы и интерфейсы:

1.1. _Card (StatelessWidget)

1.1.1. Элемент списка карточек

1.1.2. Поля:

- Text
 - Название манги
 - Строка
- DescriptionText
 - Описание манги
 - Строка
- ImageUrl
 - Ссылка на изображение
 - Строка
- Score
 - Рейтинг манги
 - Число с плавающей точкой
- onLike
 - Обработчик лайков
 - Callback
- onTap
 - Обработчик нажатия (открытия карточки)
 - Callback
- Id

- Идентификатор манги
- Строка
- IsLiked
 - Статус лайка
 - Bool

1.1.3. Методы:

- build(BuildContext context)
 - Создает визуальное отображение карточки.

1.2.SvgObjects

1.2.1.Класс, кэширующий SVG-изображения

- Метод:
 - Init()
 - Загружает SVG в кэш.

1.3. SvgRu и SvgUK

1.3.1.Наследники SvgObjects

- Метод:
 - build(BuildContext context)
 - Создает визуальное отображение конкретного SVG изображения

1.4.Body (StatefulWidget)

1.4.1. Класс отображения основной части главного экрана.

1.5._BodyState

1.5.1.Класс отображающий основную часть главного экрана. Является наследником от State(Body)

1.5.2. Поля:

- searchController

- Контроллер, ответственный за изменение текста в строке поиска.
- `scrollController`
 - Контроллер, ответственный за пролистывание карточек на главном экране

1.5.3. Методы:

- `initState()`
 - Инициализирует SVG изображения
 - Загружает данные для первой загрузки.
 - Добавляет контроллеру метод, который он будет вызывать
- `_onNextPageListener()`
 - Подгружает новые карточки при пролистывании до конца списка карточек.
- `_dispose`
 - Очищает ресурсы при удалении данного экрана из дерева виджетов.
- `_navToDetails(BuildContext context, CardData data)`
 - Открывает карточку, на которую нажали
- `_onLike(String? id, String title, bool isLiked)`
 - Вызывает событие `ChangeLikeEvent`
- `_showLiked(BuildContext context, String title, bool isLiked)`
 - Показывает уведомление снизу (Снекбар) о поставленном/убранном лайке.
- `_onRefresh()`
 - Обновляет страницу

1.6. MyHomePage (StatefulWidget)

1.6.1. Класс отображения главной страницы

1.7. _MyHomePageState

1.7.1. Основной класс отображения главной страницы. Является наследником `State<MyHomePage>`

1.7.2. Методы:

- `InitState()`
 - Создает при первом запуске приветственное сообщение
- `build(BuildContext context)`
 - Создает отображение главной страницы и вызывает `Body`

1.8. MyApp (StatelessWidget)

1.8.1. Главный класс для отображения главной страницы

1.8.2. Метод:

- `build(BuildContext context)`
 - Создает главную страницу, вызывая `MyHomePage`

Глава 3. Руководство пользователя

Приложение «Каталог манги» представляет собой инструмент для любителей манги, который предназначен для быстрого поиска и просмотра информации о нужном произведении. Это приложение разработано с учётом потребностей как «старых» поклонников манги, так и новичков, которые только начинают своё знакомство с японскими комиксами и новеллами.

Основные возможности приложения:

- Поиск манги по названию : помогает быстро найти определенное произведение.
- Просмотр информации о манге: позволяет узнать ключевую информацию о нужном произведении
- Детальная информация о манге: позволяет более подробно узнать о произведении
- Смена языка интерфейса: позволяет использовать пользователям как русским, так и иностранным
- Автоматическая подгрузка данных: удобный способ просмотра новых произведений по мере прокрутки списка.

Приложение разработано с простым и интуитивно понятным интерфейсом, что делает его доступным для пользователей любого уровня. Каталог манги — надежный инструмент в мире манге, который поможет найти новую информацию, вспомнить старые произведения и отслеживать новинки.

Целевая аудитория приложения:

- Фанаты аниме и культуры манги
- Новички и случайные читатели, желающие начать знакомство с культурой манги.
- Молодежь и подростки

Процесс запуска приложения «Каталог манги» максимально прост и не требует специальных навыков. Приложение разработано для мобильных устройств, что делает его доступным в любое время и в любом месте.

Для начала работы с приложением требуется выполнить следующие шаги:

1. Найти приложение на устройстве:

- После установки приложения на мобильное устройство его иконка появится на главном экране или в списке установленных программ.
- Иконка Каталога манги имеет узнаваемый дизайн в виде буквы «м», который, позволяет легко находить иконку приложения даже при большом количестве установленных программ.

2. Запустить приложение:

- Нажать на иконку приложения одним касанием экрана.
- При запуске приложения происходит кратковременная загрузка данных, после чего автоматически открывается главный экран приложения.

3. Первый запуск приложения:

- При запуске приложение автоматически подгрузит актуальные данные и сформирует список карточек манги на главном экране приложения.
- Для корректной работы рекомендуется обеспечить доступ к интернету, чтобы приложение могло загружать и обновлять информацию.

После успешного запуска Каталога манга пользователь сразу попадает на главный экран, где представлены карточки манги и функциональные элементы интерфейса. Главный экран разработан таким образом, чтобы пользователь мог интуитивно начать работу с приложением — будь то поиск, просмотр карточек или смена языка.

Примечание: При отсутствии подключения к интернету приложение может отображать уведомление о необходимости подключения для корректной загрузки данных.

Таким образом, запуск приложения занимает минимальное количество времени и позволяет пользователю быстро приступить к использованию его основных функций.

Главный экран является центральной частью приложения Каталог манги и предоставляет пользователю доступ ко всем ключевым функциям.

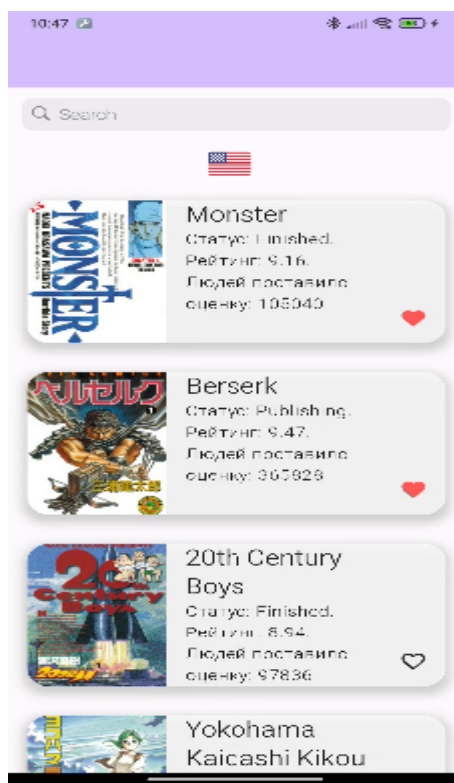


Рис. 7 Главный экран приложения

Элементы главного экрана:

1. Поисковая строка

- Используется для ввода названия для нахождения определенной манги
- Ввод текста активирует фильтрацию карточек, отображаются только те произведения, что соответствуют введенному запросу
- Пример:

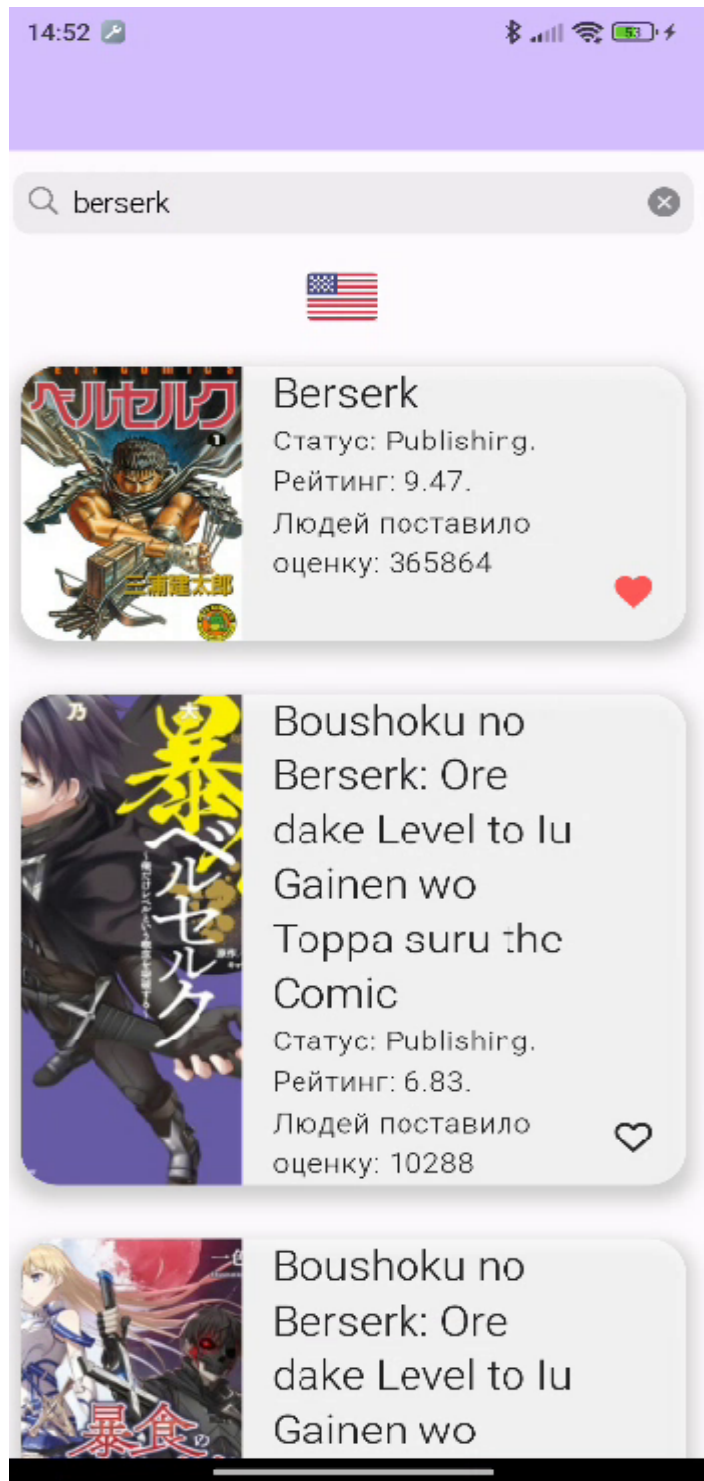


Рис 8. Запрос «Berserk»

2. Кнопка смены локализации:

- Иконка флага, расположенная под поисковой строкой, отображается флаг страны, язык которой используется
- При нажатии язык интерфейса изменяется на русский или английский
- Пример:

Если язык интерфейса был английским, то при нажатии он изменяется на русский



Рис. 9 Кнопка смены локализации

3. Список карточек манги

- Основная область экрана, расположенная под кнопкой смены локализации, в которой содержатся карточки с изображениями, описанием(статус, рейтинг, количество людей поставивших рейтинг)
- Прокрутка вниз поддерживает динамическую подгрузку, когда пользователь доходит до конца списка, новые карточки подгружаются спустя короткое время
- Элементы карточки:
 - Обложка — изображение манги
 - Описание — краткая информация о произведении
 - Кнопка лайка — позволяет добавлять мангу в понравившуюся или удалять из понравившихся

Пример:



Рис. 10 Добавление в понравившиеся



Рис. 11 Удаление из понравившихся

4. Индикатор загрузки

- Находится внизу экрана под последней карточкой на странице, выглядит как круговой индикатор загрузки при подгрузке новых данных
- Пример:

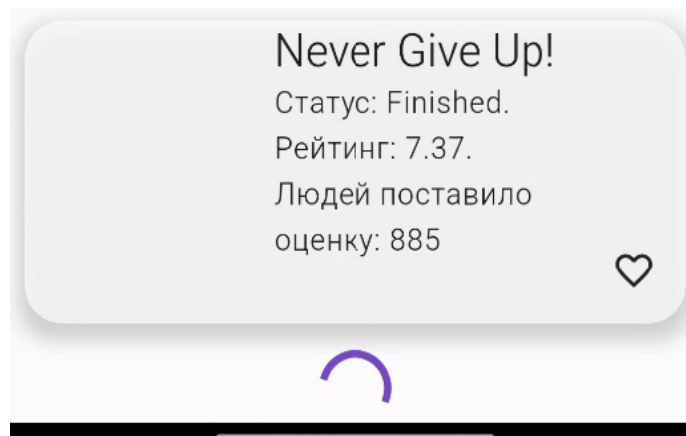


Рис. 12 Индикатор загрузки

Экран дополнительной информации открывается при нажатии на карточку манги на главном экране. Этот экран предоставляет более детальные сведения о выбранном произведении.

Элементы экрана дополнительной информации:

1. Кнопка «Назад»

- Расположена в самом верху слева, выглядит как стрелка налево
- При нажатии возвращает на главный экран приложения
- Пример:

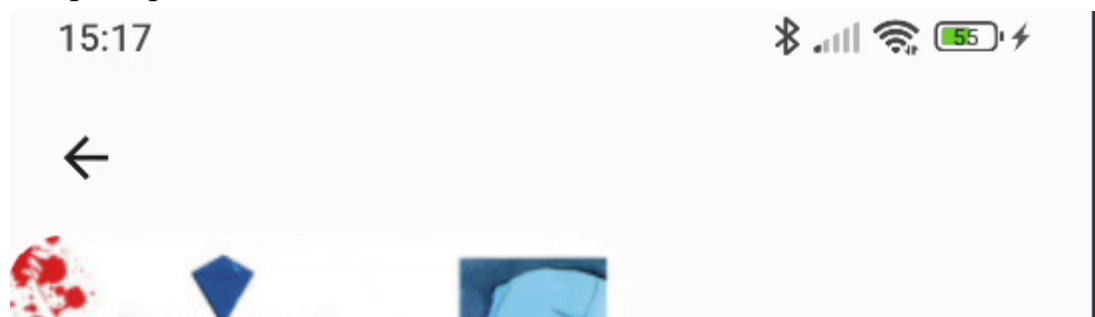


Рис. 13 Кнопка «Назад»

2. Обложка манги

- Отображается под кнопкой «Назад»
- Помогает пользователю визуально идентифицировать произведение

3. Название и статус

- Название манги выделено крупным шрифтом
- Под ним отображается статус, который позволит узнать о текущем статусе манги

4. Рейтинг и количество людей, оценивших произведение

- Рейтинг позволяет пользователю узнать о том, как люди оценили выбранное произведение
- Количество людей, оценивших произведение показывает популярность манги

Пример:

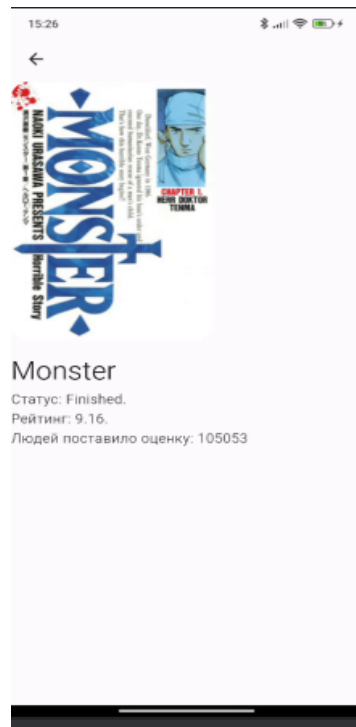


Рис. 14 Экран с подробной информацией

Контрольный пример работы с приложением

Шаг 1: Запуск приложения

- Пользователь находит иконку Каталога манги на рабочем столе мобильного устройства.
- Пользователь запускает приложение одним нажатием.

Шаг 2: Использование поиска

- Пользователь вводит названия интересующего произведения в строку поиска (например, "Berserk").
- Приложение отображает все карточки, соответствующие введённому запросу.

Шаг 3: Просмотр карточки

- Пролистав список карточек, пользователь выбирает интересующую.

- Нажатие на карточку переводит пользователя к экрану дополнительной информации.

Шаг 4: Просмотр детальной информации

- На экране дополнительной информации пользователь ознакоми́вается с изображением, описанием, рейтингом и т.д..

Шаг 5: Смена языка

- Пользователь возвращается на главный экран.
- Пользователь нажимает на иконку с изображением флага для смены языка интерфейса.

Рекомендации по использованию

Для того чтобы работа с приложением Каталог манги была максимально продуктивной, удобной и приятной, нужно следовать следующим рекомендациям:

1. Требуется стабильное подключение к интернету
 - Приложение использует интернет-соединение для загрузки карточек манги и дополнительной информации. Стабильный интернет гарантирует быструю работу поиска и плавную прокрутку карточек.
 - Рекомендуется использовать Wi-Fi соединение для экономии мобильного трафика, особенно при активном использовании приложения.
2. Использование строки поиска для ускоренного доступа
 - При большом количестве карточек манги ручной поиск может занять время.
 - Чтобы быстрее найти нужное произведение, можно использовать поисковую строку для ввода названия. Чем точнее запрос, тем быстрее будет результат.
3. Смена локализации для удобства
 - Использование функции смены языка интерфейса в зависимости от предпочтений пользователя. С помощью одного нажатия пользователь может выбрать более удобный для него язык.

- Это особенно полезно для пользователей, которые ищут мангу на английском и ищут его оригинальные названия.

4. Изучение карточек

- Каждая карточка манги содержит базовую информацию: изображение, название, краткое описание, рейтинг и популярность (в виде количества пользователей поставивших оценку). При выборе нового произведения для прочтения рекомендуется изучить все эти элементы, чтобы принять решение, стоит ли его прочитать.
- Нажатие на карточку для получения дополнительной информации, где может быть представлено расширенное описание.

5. Использование прокрутки для просмотра новых карточек

- Приложение поддерживает бесконечную прокрутку, что позволяет подгружать новые карточки манги по мере прокрутки вниз, до тех пор, пока список манги с фильтрацией или без не закончится.

Выводы

Приложение "Каталог манги" представляет собой мощный инструмент для всех любителей манги, предлагая удобный доступ к обширной библиотеке произведений различных жанров и категорий. Благодаря интуитивно понятному интерфейсу и функциональным возможностям, пользователи могут легко находить, фильтровать и наслаждаться своими любимыми мангами в любое время и в любом месте.

- В ходе работы был проведен анализ предметной области и определены требования к разработке приложения.
- Были созданы интерфейсы, которые обеспечивают интуитивную навигацию по приложению
- В ходе реализации приложения была использована современная платформа Flutter, позволяющая разрабатывать кроссплатформенные решения для Android и iOS.
- Также в ходе работы стало тестирование приложения на реальных устройствах. В результате тестирования было выявлено и устранено несколько незначительных ошибок, что позволило повысить производительность и стабильность приложения. Приложение корректно отображает данные, поддерживает ввод запросов в строку поиска и успешно переключает локализацию интерфейса.
- Разработанное приложение отвечает современным требованиям к мобильным решениям и обеспечивает:
 - Удобство взаимодействия: интуитивно понятный интерфейс и простая структура приложения;
 - Функциональность: быстрый поиск по базе данных манги и детальное отображение информации;
 - Доступность: поддержка двух языков, что делает приложение удобным для международной аудитории;

- Производительность: минимальные задержки при работе даже на устройствах со средними техническими характеристиками.

Можно сделать вывод, что были выполнены все поставленные при разработке задачи.

Список литературы

1. Разубаев С.М. [Электронный ресурс] // Гит УлГТУ : [сайт]. — URL: https://git.is.ulstu.ru/Sergey/PIbd31_Razubaev.S.M._PMU
2. Савельев Я. К. Flutter с нуля [Электронный ресурс] // Ютуб : [сайт]. — URL: https://youtube.com/playlist?list=PLEcfA4KuxR5bGM5LCzcBDkOrOX0dBZ4HN&si=0QUax8D_-GNiZCip
3. Введение в дарт. / [Электронный ресурс] // : [сайт]. — URL: <https://metanit.com/dart/tutorial/1.1.php>
4. Руководство по фреймворку Flutter / [Электронный ресурс] // : [сайт]. — URL: <https://metanit.com/dart/flutter/>
5. Онлайн компилятор Dart / [Электронный ресурс] // DartPad : [сайт]. — URL: <https://dartpad.dev/>

Исходный код

```
#Main.dart

import 'dart:io';

import 'package:flutter/material.dart';
import 'package:flutter_app/data/repositories/manga_repository.dart';
import 'package:flutter_app/presentation/home_page/bloc/bloc.dart';
import 'package:flutter_app/presentation/home_page/home_page.dart';
import 'package:flutter_app/presentation/like_bloc/like_bloc.dart';
import 'package:flutter_app/presentation/locale_bloc/locale_bloc.dart';
import 'package:flutter_app/presentation/locale_bloc/locale_state.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'components/locale/l10n/app_locale.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  // This widget is the root of your application.
  @override
  Widget build(BuildContext context) {
    return BlocProvider<LocaleBloc>(
      lazy: false,
      create: (context) =>
        LocaleBloc(Locale(Platform.localeName)),
      child: BlocBuilder<LocaleBloc, LocaleState>(
        builder: (context, state) {
          return MaterialApp(
            title: 'Flutter Demo',
            locale: state.currentLocale,
            localizationsDelegates:
              AppLocale.localizationsDelegates,
            supportedLocales:
              AppLocale.supportedLocales,
            debugShowCheckedModeBanner: false,
            theme: ThemeData(
              colorScheme:
                ColorScheme.fromSeed(seedColor:
                  Colors.deepPurple),
              useMaterial3: true,
            ),
            home:
              RepositoryProvider<MangaRepository>(
                lazy: true,
                create: (_) => MangaRepository(),
                child: BlocProvider<LikeBloc>(
                  lazy: false,
                  create: (context) => LikeBloc(),
                  child: BlocProvider<HomeBloc>(
                    lazy: false,
                    create: (context) =>
                      HomeBloc(context.read<MangaRepository>(),
                        child: const HomePage(),
                      ),
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

#app_en.arb

{
  "@@locale": "en",
  "search": "Search",
  "liked": "liked!",
  "disliked": "unliked"
}

#app_ru.arb

{
  "@@locale": "ru",
  "search": "Поиск",
  "liked": "понравился!",
  "disliked": "разонравился"
}

#context_x.dart

import 'package:flutter/widgets.dart';
import 'package:flutter_app/components/locale/l10n/app_locale.dart';

extension LocalContextX on BuildContext {
  AppLocale get locale => AppLocale.of(this!);
}

#app_locale.dart

import 'dart:async';

import 'package:flutter/foundation.dart';
import 'package:flutter/widgets.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:intl/intl.dart' as intl;

import 'app_locale_en.dart';
import 'app_locale_ru.dart';

/// Callers can lookup localized strings with an
/// instance of AppLocale
/// returned by `AppLocale.of(context)`.
///
/// Applications need to include `AppLocale.delegate()`
/// in their app's
/// `localizationDelegates` list, and the locales they
/// support in the app's
/// `supportedLocales` list. For example:
///
/// ```dart
/// import 'l10n/app_locale.dart';
///
/// return MaterialApp(
///   localizationsDelegates:
///     AppLocale.localizationsDelegates,
///   supportedLocales: AppLocale.supportedLocales,
///   home: MyApplicationHome(),
/// );
/// ```
///
/// ## Update pubspec.yaml
///
/// Please make sure to update your pubspec.yaml to
/// include the following
/// packages:
///
/// ```yaml
/// dependencies:
///   # Internationalization support.
///   flutter_localizations:
```

```

/// sdk: flutter
/// intl: any # Use the pinned version from
flutter_localizations
///
/// # Rest of dependencies
/// ...
///
/// ## iOS Applications
///
/// iOS applications define key application metadata,
including supported
/// locales, in an Info.plist file that is built into
the application bundle.
/// To configure the locales supported by your app,
you'll need to edit this
/// file.
///
/// First, open your project's ios/Runner.xcworkspace
Xcode workspace file.
/// Then, in the Project Navigator, open the Info.plist
file under the Runner
/// project's Runner folder.
///
/// Next, select the Information Property List item,
select Add Item from the
/// Editor menu, then select Localizations from the
pop-up menu.
///
/// Select and expand the newly-created Localizations
item then, for each
/// locale your application supports, add a new item
and select the locale
/// you wish to add from the pop-up menu in the Value
field. This list should
/// be consistent with the languages listed in the
AppLocale.supportedLocales
/// property.
abstract class AppLocale {
  AppLocale(String locale) : localeName =
intl.Intl.canonicalizedLocale(locale.toString());

  final String localeName;

  static AppLocale? of(BuildContext context) {
    return Localizations.of<AppLocale>(context,
AppLocale);
  }

  static const LocalizationsDelegate<AppLocale>
delegate = _AppLocaleDelegate();

  /// A list of this localizations delegate along with
the default localizations
  /// delegates.
  ///
  /// Returns a list of localizations delegates
containing this delegate along with
  /// GlobalMaterialLocalizations.delegate,
GlobalCupertinoLocalizations.delegate,
  /// and GlobalWidgetsLocalizations.delegate.
  ///
  /// Additional delegates can be added by appending to }
this list in
  /// MaterialApp. This list does not have to be used
at all if a custom list
  /// of delegates is preferred or required.
  static const List<LocalizationsDelegate<dynamic>>
localizationsDelegates =
<LocalizationsDelegate<dynamic>>[
    delegate,
    GlobalMaterialLocalizations.delegate,
    GlobalCupertinoLocalizations.delegate,
    GlobalWidgetsLocalizations.delegate,
  ];

  /// A list of this localizations delegate's supported
locales.

  static const List<Locale> supportedLocales =
<Locale>[
    Locale('en'),
    Locale('ru')
  ];

  /// No description provided for @search.
  ///
  /// In ru, this message translates to:
  /// **'Поиск'**
  String get search;

  /// No description provided for @liked.
  ///
  /// In ru, this message translates to:
  /// **'понравился! '**
  String get liked;

  /// No description provided for @disliked.
  ///
  /// In ru, this message translates to:
  /// **'разонравился'**
  String get disliked;
}

class _AppLocaleDelegate extends
LocalizationsDelegate<AppLocale> {
  const _AppLocaleDelegate();

  @override
  Future<AppLocale> load(Locale locale) {
    return
SynchronousFuture<AppLocale>(lookupAppLocale(locale));
  }

  @override
  bool isSupported(Locale locale) => <String>['en',
'ru'].contains(locale.languageCode);

  @override
  bool shouldReload(_AppLocaleDelegate old) => false;
}

AppLocale lookupAppLocale(Locale locale) {

  // Lookup logic when only language code is specified.
  switch (locale.languageCode) {
    case 'en': return AppLocaleEn();
    case 'ru': return AppLocaleRu();
  }

  throw FlutterError(
    'AppLocale.delegate failed to load unsupported
locale "$locale". This is likely '
    'an issue with the localizations generation tool.
Please file an issue '
    'on GitHub with a reproducible sample app and the
gen-l10n configuration '
    'that was used.'
  );
}

#app_locale_en.dart

import 'app_locale.dart';

/// The translations for English (`en`).
class AppLocaleEn extends AppLocale {
  AppLocaleEn([String locale = 'en']) : super(locale);

  @override
  String get search => 'Search';

  @override
  String get liked => 'liked!';

  @override

```



```

    String get disliked => 'unliked';
  }

#app_locale_ru.dart

import 'app_locale.dart';

/// The translations for Russian (`ru`).
class AppLocaleRu extends AppLocale {
  AppLocaleRu([String locale = 'ru']) : super(locale);

  @override
  String get search => 'Поиск';

  @override
  String get liked => 'понравился!';

  @override
  String get disliked => 'разонравился';
}

#debounce.dart

import 'dart:async';
import 'dart:ui';

import 'package:flutter/cupertino.dart';

class Debounce {
  factory Debounce() => _instance;

  Debounce._();

  static final Debounce _instance = Debounce._();

  static Timer? _timer;

  static void run(
    VoidCallback action, {
    Duration delay = const Duration(milliseconds: 500),
  }) {
    _timer?.cancel();
    _timer = Timer(delay, action);
  }
}

#error_callback.dart

typedef OnErrorCallback = void Function(String?
Error?);

#resources.g.dart

/// Generate by [asset_generator]
(https://github.com/fluttercandies/flutter\_asset\_genera
tor) library.
/// PLEASE DO NOT EDIT MANUALLY.
// ignore_for_file: constant_identifier_names
class R {
  const R._();

  static const String ASSETS_SVG_RU_SVG =
'assets/svg/ru.svg';

  static const String ASSETS_SVG_US_SVG =
'assets/svg/us.svg';
}

#mangas_dto.dart

import 'package:json_annotation/json_annotation.dart';

part 'mangas_dto.g.dart';

@JsonSerializable(createToJson: false)
class MangasDto {
  final List<MangaDataDto>? data;
  final MangaPaginationDto? pagination;

```

```

    const MangasDto({this.data, this.pagination});

    factory MangasDto.fromJson(Map<String, dynamic> json)
=>
      _MangasDtoFromJson(json);
  }
  @JsonSerializable(createToJson: false)
  class MangaPaginationDto {
    @JsonKey(name: 'current_page')
    final int? currentPage;
    @JsonKey(name: 'has_next_page')
    final bool? hasNextPage;
    @JsonKey(name: 'last_visible_page')
    final int? lastVisiblePage;

    const MangaPaginationDto({this.currentPage,
this.hasNextPage, this.lastVisiblePage});

    factory MangaPaginationDto.fromJson(Map<String,
dynamic> json) =>
      _MangaPaginationDtoFromJson(json);
  }

  @JsonSerializable(createToJson: false)
  class MangaDataDto {
    @JsonKey(name: "mal_id")
    final int? id;
    final String? type;
    final String? title;
    final String? status;
    final double? score;
    final int? scored_by;
    final MangaDataImagesDto? images;

    const MangaDataDto(this.title, this.status,
this.score, this.scored_by, this.images, {this.id,
this.type});

    factory MangaDataDto.fromJson(Map<String, dynamic>
json) =>
      _MangaDataDtoFromJson(json);
  }

  @JsonSerializable(createToJson: false)
  class MangaDataImagesDto {
    final MangaDataImagesJPGDto? jpg;

    const MangaDataImagesDto({this.jpg});

    factory MangaDataImagesDto.fromJson(Map<String,
dynamic> json) =>
      _MangaDataImagesDtoFromJson(json);
  }

  @JsonSerializable(createToJson: false)
  class MangaDataImagesJPGDto {
    final String? image_url;

    const MangaDataImagesJPGDto({this.image_url});

    factory MangaDataImagesJPGDto.fromJson(Map<String,
dynamic> json) =>
      _MangaDataImagesJPGDtoFromJson(json);
  }
}

#mangas_dto.g.dart

// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'mangas_dto.dart';

//
*****
*****
// JsonSerializableGenerator
//
*****
*****

```

```

MangasDto _$MangasDtoFromJson(Map<String, dynamic>
json) => MangasDto(
  data: (json['data'] as List<dynamic>?)
    ?.map((e) => MangaDataDto.fromJson(e as
Map<String, dynamic>))
    .toList(),
  pagination: json['pagination'] == null
    ? null
    : MangaPaginationDto.fromJson(
      json['pagination'] as Map<String,
dynamic>),
);

MangaPaginationDto
_$MangaPaginationDtoFromJson(Map<String, dynamic> json)
=>
  MangaPaginationDto(
    currentPage: (json['current_page'] as
num?)?.toInt(),
    hasNextPage: json['has_next_page'] as bool?,
    lastVisiblePage: (json['last_visible_page'] as
num?)?.toInt(),
  );

MangaDataDto _$MangaDataDtoFromJson(Map<String,
dynamic> json) => MangaDataDto(
  json['title'] as String?,
  json['status'] as String?,
  (json['score'] as num?)?.toDouble(),
  (json['scored_by'] as num?)?.toInt(),
  json['images'] == null
    ? null
    : MangaDataImagesDto.fromJson(json['images']
as Map<String, dynamic>),
  id: (json['mal_id'] as num?)?.toInt(),
  type: json['type'] as String?,
);

MangaDataImagesDto
_$MangaDataImagesDtoFromJson(Map<String, dynamic> json)
=>
  MangaDataImagesDto(
    jpg: json['jpg'] == null
      ? null
      : MangaDataImagesJPGDto.fromJson(json['jpg']
as Map<String, dynamic>),
  );

MangaDataImagesJPGDto _$MangaDataImagesJPGDtoFromJson(
  Map<String, dynamic> json) =>
  MangaDataImagesJPGDto(
    image_url: json['image_url'] as String?,
  );

#mangas_mapper.dart

// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'mangas_dto.dart';

//
*****
*****
// JsonSerializableGenerator
//
*****
*****

MangasDto _$MangasDtoFromJson(Map<String, dynamic>
json) => MangasDto(
  data: (json['data'] as List<dynamic>?)
    ?.map((e) => MangaDataDto.fromJson(e as
Map<String, dynamic>))
    .toList(),
  pagination: json['pagination'] == null
    ? null
    : MangaPaginationDto.fromJson(
      json['pagination'] as Map<String,
dynamic>),
);

MangaPaginationDto
_$MangaPaginationDtoFromJson(Map<String, dynamic> json)
=>
  MangaPaginationDto(
    currentPage: (json['current_page'] as
num?)?.toInt(),
    hasNextPage: json['has_next_page'] as bool?,
    lastVisiblePage: (json['last_visible_page'] as
num?)?.toInt(),
  );

MangaDataDto _$MangaDataDtoFromJson(Map<String,
dynamic> json) => MangaDataDto(
  json['title'] as String?,
  json['status'] as String?,
  (json['score'] as num?)?.toDouble(),
  (json['scored_by'] as num?)?.toInt(),
  json['images'] == null
    ? null
    : MangaDataImagesDto.fromJson(json['images']
as Map<String, dynamic>),
  id: (json['mal_id'] as num?)?.toInt(),
  type: json['type'] as String?,
);

MangaDataImagesDto
_$MangaDataImagesDtoFromJson(Map<String, dynamic> json)
=>
  MangaDataImagesDto(
    jpg: json['jpg'] == null
      ? null
      : MangaDataImagesJPGDto.fromJson(json['jpg']
as Map<String, dynamic>),
  );

MangaDataImagesJPGDto _$MangaDataImagesJPGDtoFromJson(
  Map<String, dynamic> json) =>
  MangaDataImagesJPGDto(
    image_url: json['image_url'] as String?,
  );

#api_interface.dart

import
'package:flutter_app/components/utils/error_callback.da
rt';
import
'package:flutter_app/presentation/home_page/home_page.d
art';

abstract class ApiInterface {
  Future<HomeData?> loadData({OnErrorCallback?
onError});
}

#manga_repository.dart

import
'package:flutter_app/components/utils/error_callback.da
rt';
import 'package:flutter_app/data/dtos/mangas_dto.dart';
import
'package:flutter_app/data/mappers/mangas_mapper.dart';
import
'package:flutter_app/data/repositories/api_interface.da
rt';
import 'package:dio/dio.dart';
import
'package:flutter_app/presentation/home_page/home_page.d
art';
import
'package:pretty_dio_logger/pretty_dio_logger.dart';

class MangaRepository extends ApiInterface {

```

```

static final Dio _dio = Dio()
  ..interceptors.add(PrettyDioLogger(
    requestHeader: true,
    requestBody: true,
  ));
static const String _baseUrl =
'https://api.jikan.moe';

@override
Future<HomeData?> loadData({
  OnErrorCallback? onError,
  String? q,
  int page = 1,
  int pageSize = 25,
}) async {
  try {
    const String url = '$_baseUrl/v4/manga';

    final Response<dynamic> response = await _dio
      .get<Map<dynamic, dynamic>>(url,
    queryParameters: {
      'q': q,
      'page': page,
      'limit': !(pageSize > 25) ? pageSize : 25
    });

    final MangasDto dto =
      MangasDto.fromJson(response.data as
    Map<String, dynamic>);
    final HomeData? data = dto.toDomain();
    return data;
  } on DioException catch (e) {
    onError?.call(e.response?.statusMessage);
    return null;
  }
}

#mock_repository

import
'package:flutter_app/components/utils/error_callback.dart';
import
'package:flutter_app/data/repositories/api_interface.dart';
import
'package:flutter_app/domain/models/carddata.dart';
import
'package:flutter_app/presentation/home_page/home_page.dart';

class MockRepository extends ApiInterface {
  @override
  Future<HomeData?> loadData({OnErrorCallback?
onError}) async {
    return HomeData(
      data: [
        CardData('JoJo's Bizarre Adventure',
descriptionText: 'kono dio da', imageUrl:
'https://i1.sndcdn.com/avatars-253MmMf9QZzxVBji-rvlyeg-
t1080x1080.jpg'),
        CardData('Example', descriptionText: 'what is
this?', imageUrl: 'https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcQvaBQ6nAedlqvXsh-
dLXZi2Gexy1RkDbTUKQ&s'),
        CardData('Mock data', descriptionText: 'Mock data
description', imageUrl: 'https://cdn-
user30887.skyeng.ru/uploads/6692a339c6989979804399.png'
),
      ],
    );
  }
}

#carddata.dart

```

```

class CardData {
  final String text;
  final String descriptionText;
  final String? imageUrl;
  final String? id;

  CardData(this.text,
    {required this.descriptionText,
    this.imageUrl,
    this.id});
}

#svg_objects.dart

import 'package:flutter/widgets.dart';
import 'package:flutter_svg/flutter_svg.dart';
import
'package:flutter_app/components/resources.g.dart';

abstract class SvgObjects {
  static void init() {
    final pics = <String>[
      R.ASSETS_SVG_RU_SVG,
      R.ASSETS_SVG_US_SVG,
    ];
    for (final String p in pics) {
      final loader = SvgAssetLoader(p);
      svg.cache.putIfAbsent(loader.cacheKey(null), ()
=> loader.loadBytes(null));
    }
  }
}

class SvgRu extends StatelessWidget {
  const SvgRu({super.key});

  @override
  Widget build(BuildContext context) {
    return SvgPicture.asset(R.ASSETS_SVG_RU_SVG);
  }
}

class SvgUk extends StatelessWidget {
  const SvgUk({super.key});

  @override
  Widget build(BuildContext context) {
    return SvgPicture.asset(R.ASSETS_SVG_US_SVG);
  }
}

#details_page.dart

import 'package:flutter/material.dart';
import
'package:flutter_app/domain/models/carddata.dart';

class DetailsPage extends StatelessWidget {
  final CardData data;

  const DetailsPage(this.data, {super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(),
      body: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Padding(
            padding: const EdgeInsets.only(bottom:
16.0),
            child: ClipRRect(
              borderRadius:
BorderRadius.all(Radius.circular(20)),
              child: Image.network(data.imageUrl ??
'',

```

```

    ),
  ),
  Padding(
    padding: const EdgeInsets.only(bottom:
4.0),
    child: Text(
      data.text,
      style:
Theme.of(context).textTheme.headlineLarge,
    ),
    Text(
      data.descriptionText,
      style:
Theme.of(context).textTheme.bodyLarge,
    ),
  ],
),
);
}
}

#bloc

import
'package:flutter_app/data/repositories/manga_repository
.dart';
import
'package:flutter_app/presentation/home_page/bloc/events
.dart';
import
'package:flutter_app/presentation/home_page/bloc/state.
dart';
import 'package:flutter_bloc/flutter_bloc.dart';

class HomeBloc extends Bloc<HomeEvent, HomeState>{
  final MangaRepository repo;

  HomeBloc(this.repo) : super (const HomeState()) {
    on<HomeLoadDataEvent>(_onLoadData);
  }
  Future<void> _onLoadData(HomeLoadDataEvent event,
  Emitter<HomeState> emit) async {
    if( event.nextPage == null) {
      emit(state.copyWith(isLoading: true));
    } else {
      emit(state.copyWith(isPaginationLoading: true));
    }

    final data = await repo.loadData(q: event.search,
    page: event.nextPage ?? 1);

    if (event.nextPage != null) {
      data?.data?.insertAll(0, state.data?.data ?? []);
    }

    emit(state.copyWith(data: data, isLoading: false,
    isPaginationLoading: false));
  }
}

#events.dart

abstract class HomeEvent {
  const HomeEvent();
}
class HomeLoadDataEvent extends HomeEvent {
  final String? search;
  final int? nextPage;

  const HomeLoadDataEvent({this.search,
  this.nextPage});
}

#state.dart

import 'package:equatable/equatable.dart';

```

```

import
'package:flutter_app/presentation/home_page/home_page.d
art';

class HomeState extends Equatable {
  final HomeData? data;
  final bool isLoading;
  final bool isPaginationLoading;
  const HomeState({this.data, this.isLoading = false,
  this.isPaginationLoading = false});

  HomeState copyWith({HomeData? data, bool? isLoading,
  bool? isPaginationLoading}) => HomeState(
    data: data ?? this.data, isLoading: isLoading ??
this.isLoading, isPaginationLoading:
isPaginationLoading ?? this.isPaginationLoading);

  @override
  List<Object?> get props => [data, isLoading,
  isPaginationLoading];
}

#card.data

part of 'home_page.dart';

typedef OnLikeCallback = void Function(String? id,
String title, bool isLiked?);

class _Card extends StatelessWidget {
  final String text;
  final String descriptionText;
  final String? imageUrl;
  final OnLikeCallback onLike;
  final VoidCallback? onTap;
  final String? id;
  final bool isLiked;

  const _Card(
    this.text, {
    required this.descriptionText,
    this.imageUrl,
    this.onLike,
    this.onTap,
    this.id,
    this.isLiked = false,
  });

  factory _Card.fromData(
    CardData data, {
    OnLikeCallback onLike,
    VoidCallback? onTap,
    bool isLiked = false,
  }) =>
    _Card(
      data.text,
      descriptionText: data.descriptionText,
      imageUrl: data.imageUrl,
      onLike: onLike,
      onTap: onTap,
      isLiked: isLiked,
      id: data.id,
    );

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: onTap,
      child: Container(
        margin: const EdgeInsets.all(16),
        constraints: const BoxConstraints(minHeight:
160),
        decoration: BoxDecoration(
          color: Colors.white70,
          borderRadius: BorderRadius.circular(20),
          boxShadow: [
            BoxShadow(
              color: Colors.grey.withOpacity(.5),

```



```

        bottomLeft: Radius.circular(20),
        topLeft: Radius.circular(20),
      ),
      child: SizedBox(
        height: double.infinity,
        width: 120,
        child: Stack(
          children: [
            Positioned.fill(
              child: Image.network(
                imageUrl ?? "",
                fit: BoxFit.cover,
                errorBuilder: (_, __, ___) => const Placeholder(),
              ),
            ),
          ],
        ),
      ),
      Expanded(
        child: Padding(
          padding: const EdgeInsets.only(left: 16.0),
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              Text(
                text,
                style: Theme.of(context).textTheme.headlineSmall,
              ),
              Text(
                descriptionText,
                style: Theme.of(context).textTheme.bodyLarge,
              ),
            ],
          ),
        ),
      ),
      Align(
        alignment: Alignment.bottomRight,
        child: Padding(
          padding: const EdgeInsets.only(left: 8, right: 16, bottom: 16),
          child: GestureDetector(
            onTap: () => onLike?.call(id, text, isLiked),
            child: AnimatedSwitcher(
              duration: const Duration(milliseconds: 200),
              child: isLiked
                ? const Icon(
                    Icons.favorite,
                    color: Colors.redAccent,
                    key: ValueKey<int>(0),
                  )
                : const Icon(
                    Icons.favorite_border,
                    key: ValueKey<int>(1),
                  ),
            ),
          ),
        ),
      ),
    ],
  ),
);
}
}

```

#like_bloc

```

import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_app/presentation/like_bloc/like_event.dart';
import 'package:flutter_app/presentation/like_bloc/like_state.dart';
import 'package:shared_preferences/shared_preferences.dart';

```

const String _likedPrefsKey = 'liked';

```

class LikeBloc extends Bloc<LikeEvent, LikeState> {
  LikeBloc() : super(const LikeState(likedIds: [])) {
    on<ChangeLikeEvent>(_onChangeLike);
  }
}

```

```

    on<LoadLikesEvent>(_onLoadLikes);
  }

  Future<void> _onLoadLikes(LoadLikesEvent event, Emitter<LikeState>
emit) async {
    final prefs = await SharedPreferences.getInstance();
    final data = prefs.getStringList(_likedPrefsKey);

    emit(state.copyWith(likedIds: data));
  }

  Future<void> _onChangeLike(ChangeLikeEvent event,
Emitter<LikeState> emit) async {
    final updatedList = List<String>.from(state.likedIds ?? []);

    if (updatedList.contains(event.id)) {
      updatedList.remove(event.id);
    } else {
      updatedList.add(event.id);
    }

    final prefs = await SharedPreferences.getInstance();
    prefs.setStringList(_likedPrefsKey, updatedList);

    emit(state.copyWith(likedIds: updatedList));
  }
}

#like_event.dart

abstract class LikeEvent {
  const LikeEvent();
}

class LoadLikesEvent extends LikeEvent {
  const LoadLikesEvent();
}

class ChangeLikeEvent extends LikeEvent {
  final String id;

  const ChangeLikeEvent(this.id);
}

#like_state.dart

import 'package:copy_with_extension/copy_with_extension.dart';
import 'package:equatable/equatable.dart';

part 'like_state.g.dart';

@CopyWith()
class LikeState extends Equatable {
  final List<String>? likedIds;

  const LikeState({required this.likedIds});

  @override
  List<Object?> get props => [likedIds];
}

```

#like_state.g.dart

// GENERATED CODE - DO NOT MODIFY BY HAND

part of 'like_state.dart';

```

//
*****
*****
// CopyWithGenerator
//
*****
*****

```

```

abstract class _LikeStateCWProxy {
  LikeState likedIds(List<String>? likedIds);
}

```

```

    /// This function **does support** nullification of nullable fields. All `null`
    values passed to `non-nullable` fields will be ignored. You can also use
    `LikeState(...).copyWith.fieldName(...)` to override fields one at a time with
    nullification support.

```

```

    ///
    /// Usage
    /// ```dart
    /// LikeState(...).copyWith(id: 12, name: "My name")
    /// ```
    LikeState call({
      List<String>? likedIds,
    });
  }
}

```

```

    /// Proxy class for `copyWith` functionality. This is a callable class and can
    be used as follows: `instanceOfLikeState.copyWith(...)`. Additionally
    contains functions for specific fields e.g.
    `instanceOfLikeState.copyWith.fieldName(...)`

```

```

class _$LikeStateCWProxyImpl implements _$LikeStateCWProxy {
  const _$LikeStateCWProxyImpl(this._value);

  final LikeState _value;

  @override
  LikeState likedIds(List<String>? likedIds) => this(likedIds: likedIds);

  @override

```

```

    /// This function **does support** nullification of nullable fields. All `null`
    values passed to `non-nullable` fields will be ignored. You can also use
    `LikeState(...).copyWith.fieldName(...)` to override fields one at a time with
    nullification support.

```

```

    ///
    /// Usage
    /// ```dart
    /// LikeState(...).copyWith(id: 12, name: "My name")
    /// ```
    LikeState call({
      Object? likedIds = const $CopyWithPlaceholder(),
    }) {
      return LikeState(
        likedIds: likedIds == const $CopyWithPlaceholder()
          ? _value.likedIds
          // ignore: cast_nullable_to_non_nullable
          : likedIds as List<String>?,
      );
    }
  }
}

```

```

extension $LikeStateCopyWith on LikeState {
  /// Returns a callable class that can be used as follows:
  `instanceOfLikeState.copyWith(...)` or like
  so: `instanceOfLikeState.copyWith.fieldName(...)`
  // ignore: library_private_types_in_public_api
  _$LikeStateCWProxy get copyWith => _$LikeStateCWProxyImpl(this);
}

```

```

#locale_bloc

```

```

import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:flutter_app/components/locale/110n/app_locale.dart';
import 'package:flutter_app/presentation/locale_bloc/locale_events.dart';
import 'package:flutter_app/presentation/locale_bloc/locale_state.dart';

class LocaleBloc extends Bloc<LocaleEvent, LocaleState> {
  LocaleBloc(Locale defaultLocale) : super(LocaleState(currentLocale:
    defaultLocale)) {
    on<ChangeLocaleEvent>(_onChangeLocale);
  }

  Future<void> _onChangeLocale(ChangeLocaleEvent event,
    Emitter<LocaleState> emit) async {
    final toChange = AppLocale.supportedLocales
      .firstWhere((e) => e.languageCode !=
state.currentLocale.languageCode);
    emit(state.copyWith(currentLocale: toChange));
  }
}

```

```

#locale_events

```

```

abstract class LocaleEvent {
  const LocaleEvent();
}

```

```

class ChangeLocaleEvent extends LocaleEvent {
  const ChangeLocaleEvent();
}

```

```

import 'package:copy_with_extension/copy_with_extension.dart';
import 'package:equatable/equatable.dart';
import 'package:flutter/material.dart';

```

```

part 'locale_state.g.dart';

```

```

@CopyWith()
class LocaleState extends Equatable {
  final Locale currentLocale;

  const LocaleState({required this.currentLocale});

  @override
  List<Object?> get props => [currentLocale];
}

```

```

#locale_state.g.dart

```

```

// GENERATED CODE - DO NOT MODIFY BY HAND

```

```

part of 'locale_state.dart';

```

```

//
// *****
// *****
// CopyWithGenerator
//
// *****
// *****

```

```

abstract class _$LocaleStateCWProxy {
  LocaleState currentLocale(Locale currentLocale);

  /// This function **does support** nullification of nullable fields. All `null`
  values passed to `non-nullable` fields will be ignored. You can also use
  `LocaleState(...).copyWith.fieldName(...)` to override fields one at a time
  with nullification support.

```

```

    ///
    /// Usage
    /// ```dart
    /// LocaleState(...).copyWith(id: 12, name: "My name")
    /// ```
    LocaleState call({
      Locale? currentLocale,
    });
  }
}

```

```

    /// Proxy class for `copyWith` functionality. This is a callable class and can
    be used as follows: `instanceOfLocaleState.copyWith(...)`. Additionally
    contains functions for specific fields e.g.
    `instanceOfLocaleState.copyWith.fieldName(...)`

```

```

class _$LocaleStateCWProxyImpl implements _$LocaleStateCWProxy {
  const _$LocaleStateCWProxyImpl(this._value);

  final LocaleState _value;

  @override
  LocaleState currentLocale(Locale currentLocale) =>
    this(currentLocale: currentLocale);

  @override

```

```

    /// This function **does support** nullification of nullable fields. All `null`
    values passed to `non-nullable` fields will be ignored. You can also use
    `LocaleState(...).copyWith.fieldName(...)` to override fields one at a time
    with nullification support.

```

```

    ///
    /// Usage

```

```

/// ``dart
/// LocaleState(...).copyWith(id: 12, name: "My name")
/// ````
LocaleState call({
  Object? currentLocale = const $CopyWithPlaceholder(),
}) {
  return LocaleState(
    currentLocale:
      currentLocale == const $CopyWithPlaceholder() || currentLocale ==
null
      ? _value.currentLocale
      // ignore: cast_nullable_to_non_nullable
      : currentLocale as Locale,
    );
}

extension $LocaleStateCopyWith on LocaleState {
  /// Returns a callable class that can be used as follows:
  /// `instanceOfLocaleState.copyWith(...)` or like
  /// so: `instanceOfLocaleState.copyWith.fieldName(...)`.
  // ignore: library_private_types_in_public_api
  $LocaleStateCWProxy get copyWith =>
    _$LocaleStateCWProxyImpl(this);
}

```